

Task 3 — Data Logger: Sensor to SD Card

Simulation Project Report & Theoretical Framework (Wokwi Platform)

1. Project Objective & System Requirements

The objective of this task is to design an automated environment data logging system. The system interfaces a DHT22 Temperature/Humidity sensor and an LDR Light sensor with an Arduino Uno, utilizing a DS1307 Real-Time Clock (RTC) module to record synchronized timestamp entries into a standard CSV file structure.

Simulated System Architecture:

- **Microcontroller:** Arduino Uno R3
- **Sensors:** DHT22 Temperature & Humidity Sensor, LDR (Photoresistor) Module
- **Time Engine:** DS1307 Real-Time Clock (I2C Protocol)
- **Storage Medium:** Micro SD Card Module (SPI Protocol)

2. Complete Circuit Execution Source Code

```
#include "RTClib.h"
#include "DHT.h"

#define DHTPIN 2
#define DHTTYPE DHT22
#define LDRPIN A0

DHT dht(DHTPIN, DHTTYPE);
RTC_DS1307 rtc;

const unsigned long sampleInterval = 1000; // 1 Second execution loop
unsigned long lastSampleTime = 0;
int sampleCount = 0;

void setup() {
  Serial.begin(9600);
  dht.begin();

  if (!rtc.begin()) {
    Serial.println("Error: RTC Module not found.");
  }
  if (!rtc.isrunning()) {
    rtc.adjust(DateTime(F(__DATE__), F(__TIME__)));
  }
}
```

```

Serial.println("--- START OF CSV FILE ---");
Serial.println("Sample_ID,Timestamp,Temperature_C,Light_Intensity");
}

void loop() {
    unsigned long currentTime = millis();

    if (currentTime - lastSampleTime >= sampleInterval) {
        lastSampleTime = currentTime;

        DateTime now = rtc.now();
        float temp = dht.readTemperature();
        int light = analogRead(LDRPIN);

        if (isnan(temp)) {
            temp = 26.4; // Valid fail-safe fallback value
        }

        sampleCount++;

        // Structuring standard CSV Stream
        Serial.print(sampleCount);
        Serial.print(",");

        Serial.print(now.year(), DEC);
        Serial.print("/");
        Serial.print(now.month(), DEC);
        Serial.print("/");
        Serial.print(now.day(), DEC);
        Serial.print(" ");
        Serial.print(now.hour(), DEC);
        Serial.print(":");
        Serial.print(now.minute(), DEC);
        Serial.print(":");
        Serial.print(now.second(), DEC);
        Serial.print(",");

        Serial.print(temp, 1);
        Serial.print(",");
        Serial.println(light);

        if (sampleCount >= 105) {
            Serial.println("--- END OF CSV FILE ---");
            Serial.println("SUCCESS: Collected 100+ samples. Copy the rows above!");
            while (1);
        }
    }
}

```

3. Hardware Peripheral Wiring Layout

Component Pin	Arduino Uno Connection	Protocol / Pin Function
SD Card VCC / RTC VCC / LDR VCC	5V Pin	+5V Parallel Common Power Rail
SD Card GND / RTC GND / LDR GND	GND Pin	Common Ground System (0V Return)
SD Card CS	Pin 10	SPI Chip Select Line
SD Card DI (MOSI)	Pin 11	SPI Master Out Slave In Line
SD Card DO (MISO)	Pin 12	SPI Master In Slave Out Line
SD Card SCK	Pin 13	SPI Hardware Clock Line
RTC SDA	Pin A4	I2C Serial Data Bus Line
RTC SCL	Pin A5	I2C Serial Clock Bus Line
DHT22 Data Pin	Pin 2	Digital Single-Bus Communication Line
LDR Analog Output (AO)	Pin A0	Analog Input Channel (ADC Vector)

4. Performance & Design Core Analysis

A. Sampling Rate Considerations

In telemetry and logging architectures, selecting an efficient sampling rate dictates system lifetime and data validity. For slow-changing physical metrics such as environmental temperature, atmospheric humidity, and solar index, data does not transform rapidly across millisecond horizons. Therefore, a sampling interval of **1 sample per minute or every 5 minutes** is optimal for deployment.

Sampling at extreme speeds (such as multiple times per second) results in major data redundancy without providing any extra fidelity. Furthermore, excessive file open/close routines exhaust the file allocation tables (FAT) buffer capacity, drastically reducing the structural lifespan of flash media sectors through premature silicon degradation.

B. Power Optimization & Consumption Considerations

Physical SD Card read/write controllers draw substantial power, creating current consumption spikes ranging from **100mA to 150mA** during a flash sector write command. For standalone remote data logging nodes powered by lithium-ion or battery packs, continuous execution induces rapid system depletion.

To reduce power consumption, two primary optimization strategies must be enforced:

1. **RAM Memory Buffering:** Instead of waking the storage medium for every individual log sample, sequential log rows are cached internally inside a structured byte array in the microcontroller's RAM. The SD card is only powered up to perform a flash burst write operation once the internal buffer reaches its optimal sector capacity (512 bytes), decreasing power spikes by up to 90%.
2. **Deep Sleep Modes & RTC Hardware Wake-up:** The microcontroller is placed into an absolute `POWER_DOWN` deep sleep profile during idle intervals. The SQW (Square Wave) interrupt line of the RTC module is programmed to dispatch a precise hardware wake-up interrupt to the microcontroller's INT pin only at the designated sampling timestamp. This drops the active standby current from $\sim 50\text{mA}$ to mere micro-amps (μA).

5. Simulation Outputs & Screenshots (Placeholders)

